JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Deflater

java.lang.Object
    java.util.zip.Deflater

All Implemented Interfaces:

AutoCloseable

```
public class Deflater
extends Object
implements AutoCloseable
```

This class provides support for general purpose compression using the popular ZLIB compression library. The ZLIB compression library was initially developed as part of the PNG graphics standard and is not protected by patents. It is fully described in the specifications at the java.util.zip package description.

Unless otherwise noted, passing a `null` argument to a method in this class will cause a `NullPointerException` to be thrown.

This class deflates sequences of bytes into ZLIB compressed data format. The input byte sequence is provided in either a byte array or a `ByteBuffer`, via one of the `setInput()` methods. The output byte sequence is written to the output byte array or `ByteBuffer` passed to the `deflate()` methods.

To release the resources used by a `Deflater`, an application must close it by invoking its `end()` or `close()` method.

API Note:

This class implements `AutoCloseable` to facilitate its usage with `try`-with-resources statement. The close() method simply calls `end()`.

The following code fragment demonstrates a trivial compression and decompression of a string using `Deflater` and `Inflater`.

```
// Encode a String into bytes
String inputString = "blahblahblah\u20AC\u20AC";
byte[] input = inputString.getBytes(StandardCharsets.UTF_8);

// Compress the bytes
ByteArrayOutputStream compressedBaos = new ByteArrayOutputStream();
try (Deflater compressor = new Deflater()) {
    compressor.setInput(input);
    // Let the compressor know that the complete input
    // has been made available
    compressor.finish();
```

```
    // Keep compressing the input till the compressor
    // is finished compressing
    while (!compressor.finished()) {
        // Use some reasonable size for the temporary buffer
        // based on the data being compressed
        byte[] tmpBuffer = new byte[100];
        int numCompressed = compressor.deflate(tmpBuffer);
        // Copy over the compressed bytes from the temporary
        // buffer into the final byte array
        compressedBaos.write(tmpBuffer, 0, numCompressed);
    }
}

// Decompress the bytes
ByteArrayOutputStream decompressedBaos = new ByteArrayOutputStream();
try (Inflater decompressor = new Inflater()) {
    byte[] compressed = compressedBaos.toByteArray();
    decompressor.setInput(compressed, 0, compressed.length);
    while (!decompressor.finished()) {
        // Use some reasonable size for the temporary buffer,
        // based on the data being decompressed; in this example,
        // we use a small buffer size
        byte[] tmpBuffer = new byte[100];
        int numDecompressed = 0;
        try {
            numDecompressed = decompressor.inflate(tmpBuffer);
        } catch (DataFormatException dfe) {
            // Handle the exception suitably, in this example
            // we just rethrow it
            throw new RuntimeException(dfe);
        }
        // Copy over the decompressed bytes from the temporary
        // buffer into the final byte array
        decompressedBaos.write(tmpBuffer, 0, numDecompressed);
    }
}
// Decode the bytes into a String
String outputString = decompressedBaos.toString(StandardCharsets.UTF_8);
```

Since:

1.1

See Also:

Inflater

Field Summary

Fields

Modifier and Type	Field	Description
static final int	BEST_COMPRESSION	Compression level for best compression.
static final int	BEST_SPEED	Compression level for fastest compression.
static final int	DEFAULT_COMPRESSION	Default compression level.
static final int	DEFAULT_STRATEGY	Default compression strategy.
static final int	DEFLATED	Compression method for the deflate algorithm (the only one currently supported).
static final int	FILTERED	Compression strategy best used for data consisting mostly of small values with a somewhat random distribution.
static final int	FULL_FLUSH	Compression flush mode used to flush out all pending output and reset the deflater.
static final int	HUFFMAN_ONLY	Compression strategy for Huffman coding only.
static final int	NO_COMPRESSION	Compression level for no compression.
static final int	NO_FLUSH	Compression flush mode used to achieve best compression result.

`static final int` **SYNC_FLUSH**	Compression flush mode used to flush out all pending output; may degrade compression for some compression algorithms.

Constructor Summary

Constructors

Constructor	Description
Deflater()	Creates a new compressor with the default compression level.
Deflater(int level)	Creates a new compressor using the specified compression level.
Deflater(int level, boolean nowrap)	Creates a new compressor using the specified compression level.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	**close**()	Closes and releases the resources held by this `Deflater` and discards any unprocessed input.
int	**deflate**(byte[] output)	Compresses the input data and fills specified buffer with compressed data.
int	**deflate**(byte[] output, int off, int len)	Compresses the input data and fills specified buffer with compressed data.
int	**deflate**(byte[] output, int off, int len, int flush)	Compresses the input data and fills the

		specified buffer with compressed data.
int	**deflate**(ByteBuffer output)	Compresses the input data and fills specified buffer with compressed data.
int	**deflate**(ByteBuffer output, int flush)	Compresses the input data and fills the specified buffer with compressed data.
void	**end**()	Closes and releases the resources held by this `Deflater` and discards any unprocessed input.
void	**finish**()	When called, indicates that compression should end with the current contents of the input buffer.
boolean	**finished**()	Returns true if the end of the compressed data output stream has been reached.
int	**getAdler**()	Returns the ADLER-32 value of the uncompressed data.
long	**getBytesRead**()	Returns the total number of uncompressed bytes input so far.
long	**getBytesWritten**()	Returns the total number of compressed bytes output so far.
int	**getTotalIn**()	**Deprecated.**

		Use `getBytesRead()` instead
int	**`getTotalOut`**`()`	**Deprecated.** Use `getBytesWritten()` instead
boolean	**`needsInput`**`()`	Returns true if no data remains in the input buffer.
void	**`reset`**`()`	Resets deflater so that a new set of input data can be processed.
void	**`setDictionary`**`(byte[] dictionary)`	Sets preset dictionary for compression.
void	**`setDictionary`**`(byte[] dictionary, int off, int len)`	Sets preset dictionary for compression.
void	**`setDictionary`**`(ByteBuffer dictionary)`	Sets preset dictionary for compression.
void	**`setInput`**`(byte[] input)`	Sets input data for compression.
void	**`setInput`**`(byte[] input, int off, int len)`	Sets input data for compression.
void	**`setInput`**`(ByteBuffer input)`	Sets input data for compression.
void	**`setLevel`**`(int level)`	Sets the compression level to the specified value.
void	**`setStrategy`**`(int strategy)`	Sets the compression strategy to the specified value.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Field Details

DEFLATED

`public static final int DEFLATED`

Compression method for the deflate algorithm (the only one currently supported).

See Also:

Constant Field Values

NO_COMPRESSION

`public static final int NO_COMPRESSION`

Compression level for no compression.

See Also:

Constant Field Values

BEST_SPEED

`public static final int BEST_SPEED`

Compression level for fastest compression.

See Also:

Constant Field Values

BEST_COMPRESSION

`public static final int BEST_COMPRESSION`

Compression level for best compression.

See Also:

Constant Field Values

DEFAULT_COMPRESSION

`public static final int DEFAULT_COMPRESSION`

Default compression level.

See Also:

Constant Field Values

FILTERED

`public static final int FILTERED`

Compression strategy best used for data consisting mostly of small values with a somewhat random distribution. Forces more Huffman coding and less string matching.

See Also:

Constant Field Values

HUFFMAN_ONLY

`public static final int HUFFMAN_ONLY`

Compression strategy for Huffman coding only.

See Also:

Constant Field Values

DEFAULT_STRATEGY

`public static final int DEFAULT_STRATEGY`

Default compression strategy.

See Also:

Constant Field Values

NO_FLUSH

`public static final int NO_FLUSH`

Compression flush mode used to achieve best compression result.

Since:

1.7

See Also:

`deflate(byte[], int, int, int)`,
Constant Field Values

SYNC_FLUSH

`public static final int SYNC_FLUSH`

Compression flush mode used to flush out all pending output; may degrade compression for some compression algorithms.

Since:

1.7

See Also:

`deflate(byte[], int, int, int)`,
Constant Field Values

FULL_FLUSH

```
public static final int FULL_FLUSH
```

Compression flush mode used to flush out all pending output and reset the deflater. Using this mode too often can seriously degrade compression.

Since:

1.7

See Also:

`deflate(byte[], int, int, int)`,
Constant Field Values

Constructor Details

Deflater

```
public Deflater(int level,
                boolean nowrap)
```

Creates a new compressor using the specified compression level. If 'nowrap' is true then the ZLIB header and checksum fields will not be used in order to support the compression format used in both GZIP and PKZIP.

Parameters:

`level` - the compression level (0-9)

`nowrap` - if true then use GZIP compatible compression

Deflater

```
public Deflater(int level)
```

Creates a new compressor using the specified compression level. Compressed data will be generated in ZLIB format.

Parameters:

`level` - the compression level (0-9)

Deflater

```
public Deflater()
```

Creates a new compressor with the default compression level. Compressed data will be generated in ZLIB format.

Method Details

setInput

```
public void setInput(byte[] input,
                     int off,
                     int len)
```

Sets input data for compression.

One of the `setInput()` methods should be called whenever `needsInput()` returns true indicating that more input data is required.

Parameters:

`input` - the input data bytes

`off` - the start offset of the data

`len` - the length of the data

See Also:

`needsInput()`

setInput

```
public void setInput(byte[] input)
```

Sets input data for compression.

One of the `setInput()` methods should be called whenever `needsInput()` returns true indicating that more input data is required.

Parameters:

`input` - the input data bytes

See Also:

`needsInput()`

setInput

```
public void setInput(ByteBuffer input)
```

Sets input data for compression.

One of the `setInput()` methods should be called whenever `needsInput()` returns true indicating that more input data is required.

The given buffer's position will be advanced as deflate operations are performed, up to the buffer's limit. The input buffer may be modified (refilled) between deflate operations; doing so is equivalent to creating a new buffer and setting it with this method.

Modifying the input buffer's contents, position, or limit concurrently with an deflate operation will result in undefined behavior, which may include incorrect operation results or operation failure.

Parameters:

`input` - the input data bytes

Since:

11

See Also:

`needsInput()`

setDictionary

```
public void setDictionary(byte[] dictionary,
                          int off,
                          int len)
```

Sets preset dictionary for compression. A preset dictionary is used when the history buffer can be predetermined. When the data is later uncompressed with Inflater.inflate(), Inflater.getAdler() can be called in order to get the Adler-32 value of the dictionary required for decompression.

Parameters:

`dictionary` - the dictionary data bytes

`off` - the start offset of the data

`len` - the length of the data

Throws:

`IllegalStateException` - if the Deflater is closed

See Also:

`Inflater.inflate(byte[], int, int)`,
`Inflater.getAdler()`

setDictionary

```
public void setDictionary(byte[] dictionary)
```

Sets preset dictionary for compression. A preset dictionary is used when the history buffer can be predetermined. When the data is later uncompressed with Inflater.inflate(), Inflater.getAdler() can be called in order to get the Adler-32 value of the dictionary required for decompression.

Parameters:

`dictionary` - the dictionary data bytes

Throws:

`IllegalStateException` - if the Deflater is closed

See Also:

`Inflater.inflate(byte[], int, int)`,
`Inflater.getAdler()`

setDictionary

```
public void setDictionary(ByteBuffer dictionary)
```

Sets preset dictionary for compression. A preset dictionary is used when the history buffer can be predetermined. When the data is later uncompressed with Inflater.inflate(), Inflater.getAdler() can be called in order to get the Adler-32 value of the dictionary required for decompression.

The bytes in given byte buffer will be fully consumed by this method. On return, its position will equal its limit.

Parameters:

`dictionary` - the dictionary data bytes

Throws:

`IllegalStateException` - if the Deflater is closed

Since:

11

See Also:

`Inflater.inflate(byte[], int, int)`,
`Inflater.getAdler()`

setStrategy

```
public void setStrategy(int strategy)
```

Sets the compression strategy to the specified value.

If the compression strategy is changed, the next invocation of `deflate` will compress the input available so far with the old strategy (and may be flushed); the new strategy will take effect only after that invocation.

Parameters:

`strategy` - the new compression strategy

Throws:

`IllegalArgumentException` - if the compression strategy is invalid

setLevel

```
public void setLevel(int level)
```

Sets the compression level to the specified value.

If the compression level is changed, the next invocation of `deflate` will compress the input available so far with the old level (and may be flushed); the new level will take effect only after that invocation.

Parameters:

`level` - the new compression level (0-9)

Throws:

`IllegalArgumentException` - if the compression level is invalid

needsInput

```
public boolean needsInput()
```

Returns true if no data remains in the input buffer. This can be used to determine if one of the `setInput()` methods should be called in order to provide more input.

Returns:

true if the input data buffer is empty and setInput() should be called in order to provide more input

finish

```
public void finish()
```

When called, indicates that compression should end with the current contents of the input buffer.

finished

```
public boolean finished()
```

Returns true if the end of the compressed data output stream has been reached.

Returns:

true if the end of the compressed data output stream has been reached

deflate

```
public int deflate(byte[] output,
                   int off,
                   int len)
```

Compresses the input data and fills specified buffer with compressed data. Returns actual number of bytes of compressed data. A return value of 0 indicates that `needsInput` should be called in order to determine if more input data is required.

This method uses `NO_FLUSH` as its compression flush mode. An invocation of this method of the form `deflater.deflate(b, off, len)` yields the same result as the invocation of `deflater.deflate(b, off, len, Deflater.NO_FLUSH)`.

Parameters:

`output` - the buffer for the compressed data

`off` - the start offset of the data

`len` - the maximum number of bytes of compressed data

Returns:

the actual number of bytes of compressed data written to the output buffer

Throws:

`IllegalStateException` - if the Deflater is closed

deflate

```
public int deflate(byte[] output)
```

Compresses the input data and fills specified buffer with compressed data. Returns actual number of bytes of compressed data. A return value of 0 indicates that `needsInput` should be called in order to determine if more input data is required.

This method uses `NO_FLUSH` as its compression flush mode. An invocation of this method of the form `deflater.deflate(b)` yields the same result as the invocation of `deflater.deflate(b, 0, b.length, Deflater.NO_FLUSH)`.

Parameters:

`output` - the buffer for the compressed data

Returns:

the actual number of bytes of compressed data written to the output buffer

Throws:

`IllegalStateException` - if the Deflater is closed

deflate

```
public int deflate(ByteBuffer output)
```

Compresses the input data and fills specified buffer with compressed data. Returns actual number of bytes of compressed data. A return value of 0 indicates that `needsInput` should be called in order to determine if more input data is required.

This method uses `NO_FLUSH` as its compression flush mode. An invocation of this method of the form `deflater.deflate(output)` yields the same result as the invocation of `deflater.deflate(output, Deflater.NO_FLUSH)`.

Parameters:

`output` - the buffer for the compressed data

Returns:

the actual number of bytes of compressed data written to the output buffer

Throws:

`ReadOnlyBufferException` - if the given output buffer is read-only

`IllegalStateException` - if the Deflater is closed

Since:

11

deflate

```
public int deflate(byte[] output,
                   int off,
                   int len,
                   int flush)
```

Compresses the input data and fills the specified buffer with compressed data. Returns actual number of bytes of data compressed.

Compression flush mode is one of the following three modes:

- `NO_FLUSH`: allows the deflater to decide how much data to accumulate, before producing output, in order to achieve the best compression (should be used in normal use scenario). A return value of 0 in this flush mode indicates that `needsInput()` should be called in order to determine if more input data is required.
- `SYNC_FLUSH`: all pending output in the deflater is flushed, to the specified output buffer, so that an inflater that works on compressed data can get all input data available so far (In particular the `needsInput()` returns `true` after this invocation if enough output space is provided). Flushing

with SYNC_FLUSH may degrade compression for some compression algorithms and so it should be used only when necessary.
- FULL_FLUSH: all pending output is flushed out as with SYNC_FLUSH. The compression state is reset so that the inflater that works on the compressed output data can restart from this point if previous compressed data has been damaged or if random access is desired. Using FULL_FLUSH too often can seriously degrade compression.

In the case of FULL_FLUSH or SYNC_FLUSH, if the return value is len, the space available in output buffer b, this method should be invoked again with the same flush parameter and more output space. Make sure that len is greater than 6 to avoid flush marker (5 bytes) being repeatedly output to the output buffer every time this method is invoked.

If the setInput(ByteBuffer) method was called to provide a buffer for input, the input buffer's position will be advanced by the number of bytes consumed by this operation.

Parameters:

output - the buffer for the compressed data

off - the start offset of the data

len - the maximum number of bytes of compressed data

flush - the compression flush mode

Returns:

the actual number of bytes of compressed data written to the output buffer

Throws:

IllegalArgumentException - if the flush mode is invalid

IllegalStateException - if the Deflater is closed

Since:

1.7

deflate

```
public int deflate(ByteBuffer output,
                   int flush)
```

Compresses the input data and fills the specified buffer with compressed data. Returns actual number of bytes of data compressed.

Compression flush mode is one of the following three modes:

- NO_FLUSH: allows the deflater to decide how much data to accumulate, before producing output, in order to achieve the best compression

(should be used in normal use scenario). A return value of 0 in this flush mode indicates that `needsInput()` should be called in order to determine if more input data is required.

- `SYNC_FLUSH`: all pending output in the deflater is flushed, to the specified output buffer, so that an inflater that works on compressed data can get all input data available so far (In particular the `needsInput()` returns `true` after this invocation if enough output space is provided). Flushing with `SYNC_FLUSH` may degrade compression for some compression algorithms and so it should be used only when necessary.
- `FULL_FLUSH`: all pending output is flushed out as with `SYNC_FLUSH`. The compression state is reset so that the inflater that works on the compressed output data can restart from this point if previous compressed data has been damaged or if random access is desired. Using `FULL_FLUSH` too often can seriously degrade compression.

In the case of `FULL_FLUSH` or `SYNC_FLUSH`, if the return value is equal to the remaining space of the buffer, this method should be invoked again with the same `flush` parameter and more output space. Make sure that the buffer has at least 6 bytes of remaining space to avoid the flush marker (5 bytes) being repeatedly output to the output buffer every time this method is invoked.

On success, the position of the given `output` byte buffer will be advanced by as many bytes as were produced by the operation, which is equal to the number returned by this method.

If the `setInput(ByteBuffer)` method was called to provide a buffer for input, the input buffer's position will be advanced by the number of bytes consumed by this operation.

Parameters:

`output` - the buffer for the compressed data

`flush` - the compression flush mode

Returns:

the actual number of bytes of compressed data written to the output buffer

Throws:

`IllegalArgumentException` - if the flush mode is invalid

`ReadOnlyBufferException` - if the given output buffer is read-only

`IllegalStateException` - if the Deflater is closed

Since:

11

getAdler

```
public int getAdler()
```

Returns the ADLER-32 value of the uncompressed data.

Returns:

the ADLER-32 value of the uncompressed data

Throws:

`IllegalStateException` - if the Deflater is closed

getTotalIn

```
@Deprecated(since="23")
public int getTotalIn()
```

Deprecated.
Use `getBytesRead()` instead

Returns the total number of uncompressed bytes input so far.

Implementation Requirements:

This method returns the equivalent of `(int) getBytesRead()` and therefore cannot return the correct value when it is greater than `Integer.MAX_VALUE`.

Returns:

the total number of uncompressed bytes input so far

Throws:

`IllegalStateException` - if the Deflater is closed

getBytesRead

```
public long getBytesRead()
```

Returns the total number of uncompressed bytes input so far.

Returns:

the total (non-negative) number of uncompressed bytes input so far

Throws:

`IllegalStateException` - if the Deflater is closed

Since:

1.5

getTotalOut

```
@Deprecated(since="23")
public int getTotalOut()
```

Deprecated.
Use `getBytesWritten()` instead

Returns the total number of compressed bytes output so far.

Implementation Requirements:

This method returns the equivalent of `(int) getBytesWritten()` and therefore cannot return the correct value when it is greater than `Integer.MAX_VALUE`.

Returns:

the total number of compressed bytes output so far

Throws:

`IllegalStateException` - if the Deflater is closed

getBytesWritten

```
public long getBytesWritten()
```

Returns the total number of compressed bytes output so far.

Returns:

the total (non-negative) number of compressed bytes output so far

Throws:

`IllegalStateException` - if the Deflater is closed

Since:

1.5

reset

```
public void reset()
```

Resets deflater so that a new set of input data can be processed. Keeps current compression level and strategy settings.

Throws:

`IllegalStateException` - if the Deflater is closed

end

```
public void end()
```

Closes and releases the resources held by this `Deflater` and discards any unprocessed input.

If the `Deflater` is already closed then invoking this method has no effect.

Implementation Requirements:

Subclasses should override this method to clean up the resources acquired by the subclass.

See Also:

`close()`

close

```
public void close()
```

Closes and releases the resources held by this `Deflater` and discards any unprocessed input.

Specified by:

`close` in interface `AutoCloseable`

Implementation Requirements:

This method calls the `end()` method.

Since:

25

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.